01

Was ist Cloud-native AI?

AI Engineering · Anwendungen planen · AI Engineering Stack · Cloud-native

Cloud-native AI · B.Sc. Angewandte Informatik · HTWG Konstanz · WS 2026/27 · Grundlage: Huyen, AI Engineering, Kap. 1 · CNCF, Cloud Native AI Whitepaper

Lernziele

- ✓ Erklären, was AI Engineering ist und worin es sich von klassischem ML Engineering unterscheidet.
- ✓ Für eine AI-Anwendung begründen, ob und wie man sie baut: Rolle der KI, Rolle des Menschen, Nützlichkeitsschwelle.
- ✓ Die drei Schichten des AI Engineering Stacks benennen und Aufgaben zuordnen.
- ✓ Erklären, was cloud-native bedeutet und warum AI-Workloads diese Prinzipien herausfordern.
- ✓ Die Zielarchitektur des Semesters beschreiben.

Worum es geht

Ihr habt sicher schon einmal ein Sprachmodell über eine API aufgerufen oder in einem Notebook ausprobiert. Eine Frage hinein, eine Antwort heraus, nach einer halben Stunde steht eine beeindruckende Demo. Genau hier beginnt das Problem, um das sich dieser Kurs dreht: **Eine gute Demo ist noch kein gutes Produkt.**

Am deutlichsten sieht man das Muster beim autonomen Fahren. Ein Shuttle, das auf einer abgesperrten Strecke bei gutem Wetter fährt, ist vergleichsweise schnell gebaut. Dann kommen Regen, Baustellen, Kinder auf dem Gehweg, blendende Sonne oder ein Lieferwagen in zweiter Reihe. Jeder dieser Sonderfälle ist für sich selten, zusammen machen sie aber den Großteil der Arbeit aus.



Abbildung 1: Autonomes Shuttle von ZF. Die Demo auf abgesperrter Strecke ist der leichte Teil.

Bei AI-Anwendungen ist es genauso. Huyen zitiert dazu eine Erfahrung von LinkedIn: Das Team erreichte in einem Monat etwa 80 Prozent der gewünschten Qualität. Dieser schnelle Erfolg verleitete dazu, den Rest zu unterschätzen. Bis das Produkt über 95 Prozent lag, vergingen weitere vier Monate, vor allem mit Sonderfällen und Halluzinationen. Andere Autoren beschreiben dasselbe Muster: Von 0 auf 60 geht es leicht, von 60 auf 100 wird es mühsam.

Was fehlt zwischen Demo und Produkt? Ein AI-System im Betrieb muss Wissen abrufen, das das Modell nicht gelernt hat. Es muss Werkzeuge nutzen, gemessen werden, in Containern laufen, skalieren, überwacht und sicher aktualisiert werden. Dieses Kapitel gibt euch die Landkarte dafür. Die einzelnen Schichten bauen wir in den kommenden Wochen auf.

> ★ Es ist leicht, mit Foundation Models eine beeindruckende Demo zu bauen. Schwer ist der Weg vom Prototyp in einen zuverlässigen Betrieb. Dieser Weg ist der Inhalt des Kurses.

Von Foundation Models zu AI Engineering

Was ein Foundation Model ist

Sprachmodelle gibt es seit Jahrzehnten. Der Sprung zu den heutigen Modellen kam durch **Self-Supervision**: Ein Modell lernt, das nächste Wort eines Textes vorherzusagen. Die Trainingsdaten müssen dafür nicht von Menschen beschriftet werden, die Beschriftung steckt im Text selbst. Dadurch lassen sich Modelle auf riesigen Textmengen trainieren und werden zu **Large Language Models**.

Werden solche Modelle zusätzlich mit Bildern, Audio oder Video trainiert, spricht man von **Foundation Models**. Der Name sagt, wofür sie gedacht sind: als Fundament, auf dem man viele verschiedene Anwendungen baut.

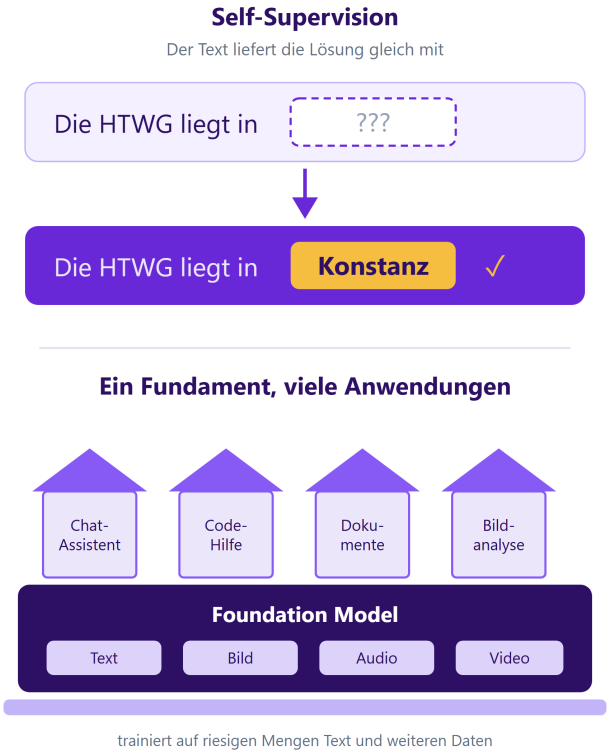


Abbildung 2: Oben: Self-Supervision, der Text liefert die Lösung gleich mit. Unten: ein Foundation Model als Fundament für viele Anwendungen.

Wie diese Modelle intern funktionieren und warum sie manchmal halluzinieren, ist Thema von VL 2.

AI Engineering

> **AI Engineering –** Der Prozess, Anwendungen auf Basis von Foundation Models zu bauen.

Anwendungen mit maschinellem Lernen gibt es seit über zehn Jahren, meist unter dem Namen ML Engineering oder MLOps. Warum also ein neuer Begriff? Huyen nennt drei Faktoren, die zusammen AI Engineering zu einer eigenen, schnell wachsenden Disziplin gemacht haben:

1. **Allgemeine Fähigkeiten:** Foundation Models können nicht nur bekannte Aufgaben besser lösen, sondern viel mehr Aufgaben überhaupt erst lösen. Damit wächst die Zahl sinnvoller Anwendungen enorm.
2. **Mehr Investitionen:** Weil AI-Anwendungen billiger und schneller zu bauen sind, lohnen sie sich häufiger. Unternehmen investieren entsprechend.

3. **Niedrige Einstiegshürde:** Modelle stehen als Dienst über eine API bereit (**Model as a Service**). Man braucht keine eigene Infrastruktur, um ein starkes Modell zu nutzen.

AI Engineering ist nicht ML Engineering

Der Unterschied zum klassischen ML Engineering lässt sich in drei Punkten fassen:

Punkt	Klassisches ML Engineering	AI Engineering
Modell	Ihr trainiert euer eigenes Modell	Ihr nutzt ein Modell, das jemand anderes trainiert hat, und **passt es an**
Ressourcen	Modelle sind klein, laufen oft auf CPUs	Modelle sind groß, brauchen GPUs und haben höhere Latenz. Inferenzoptimierung wird wichtiger
Ausgaben	Geschlossen: „Spam“ oder „kein Spam“	Offen: beliebiger Text. Das macht **Evaluation** deutlich schwerer

Für das Anpassen eines Modells gibt es zwei Familien von Techniken. **Promptbasierte Techniken** wie Prompt Engineering ändern das Modell nicht, sondern geben ihm Anweisungen und Kontext. Sie sind schnell, brauchen wenig Daten und reichen für viele Anwendungen. **Finetuning** verändert die Gewichte des Modells. Das ist aufwendiger, kann aber Qualität, Latenz und Kosten deutlich verbessern. In diesem Kurs arbeiten wir fast ausschließlich mit promptbasierten Techniken und dem Kontext, den wir dem Modell geben.

> ⚠ **Achtung:** Wer einem Modell Beispiele in den Prompt schreibt, **trainiert** es nicht. Das Modell bleibt unverändert, es bekommt nur mehr Kontext. Training heißt immer: Die Gewichte ändern sich.

AI-Anwendungen planen

Bevor man eine AI-Anwendung baut, lohnen sich ein paar Fragen. Sie entscheiden später über Architektur und Aufwand.

Welche Rolle spielt die KI?

Huyen greift drei Unterscheidungen aus Apples Richtlinien für KI-Produkte auf:

- **Kritisch oder ergänzend:** Funktioniert die Anwendung auch ohne KI? Face ID funktioniert ohne Gesichtserkennung nicht, Gmail funktioniert auch ohne Formulierungsvorschläge. Je kritischer die KI, desto zuverlässiger muss sie sein.

- **Reaktiv oder proaktiv:** Ein Chatbot antwortet auf eine Anfrage und muss schnell sein. Ein proaktiver Hinweis kann vorab berechnet werden, muss dafür aber sehr gut sein, weil niemand danach gefragt hat.
- **Dynamisch oder statisch:** Lernt die Funktion laufend aus Nutzerverhalten, oder wird sie nur mit neuen Versionen aktualisiert?

Welche Rolle spielt der Mensch?

Eine KI kann Menschen Vorschläge machen, einfache Fälle selbst erledigen und schwierige weiterreichen, oder alles allein entscheiden. Menschen in den Entscheidungsprozess einzubinden heißt **Human-in-the-Loop**. Microsoft beschreibt einen schrittweisen Weg dorthin, **Crawl, Walk, Run**: Zuerst entscheidet immer ein Mensch, dann darf die KI intern direkt handeln, zuletzt auch gegenüber externen Nutzern.

Wann ist die Anwendung gut genug?

Bevor eine Anwendung zu Nutzern kommt, braucht sie eine klare **Nützlichkeitsschwelle**. Huyen nennt vier Gruppen von Kennzahlen:

Gruppe	Beispiele
Qualität	Wie gut und wie korrekt sind die Antworten?
Latenz	**Time to First Token** (TTFT): Wie lange bis zum ersten Zeichen? **Time per Output Token** (TPOT): Wie schnell kommen die weiteren? Dazu die Gesamtdauer
Kosten	Was kostet eine Anfrage?
Weiteres	Nachvollziehbarkeit, Fairness

Diese Kennzahlen begleiten uns durch das ganze Semester. Qualität messen wir in VL 8, Latenz und Kosten in VL 10, und in der heutigen Übung messt ihr eure erste TTFT.

Wartung

Eine AI-Anwendung ist nie fertig. Modelle werden besser und billiger, Anbieter ändern Preise oder verschwinden, Regulierung kommt hinzu. Weil die Anbieter ihre Schnittstellen angleichen, lässt sich ein Modell heute oft gegen ein anderes austauschen. Prompts und Evaluation müsst ihr trotzdem anpassen, denn jedes Modell hat seine Eigenheiten. Ohne Versionierung und Evaluation wird jeder Wechsel zum Risiko.

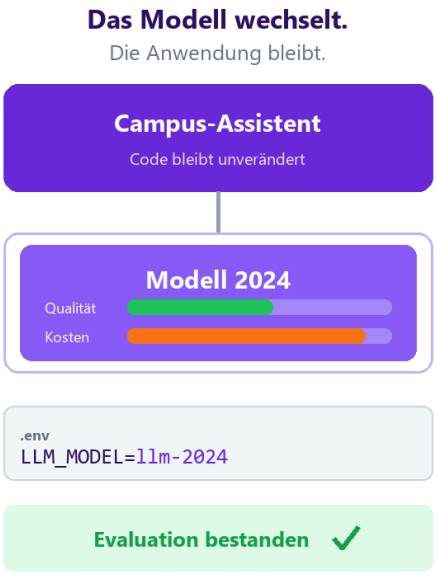


Abbildung 3: Das Modell wechselt, die Anwendung bleibt. Im Code ändert sich nichts, nur die Konfiguration. Bevor das neue Modell live geht, muss die Evaluation bestanden sein.

> 💡 **Tipp:** Baut euer System von Anfang an so, dass das Modell austauschbar ist: Adresse, Name und Schlüssel des Modells stehen in der Konfiguration, nicht im Code. Genau das übt ihr heute.

Der AI Engineering Stack

Die Zahl neuer Werkzeuge und Modelle ist überwältigend. Statt jedem Trend hinterherzulaufen, hilft eine stabile Gliederung. Huyen teilt jede AI-Anwendung in drei Schichten:

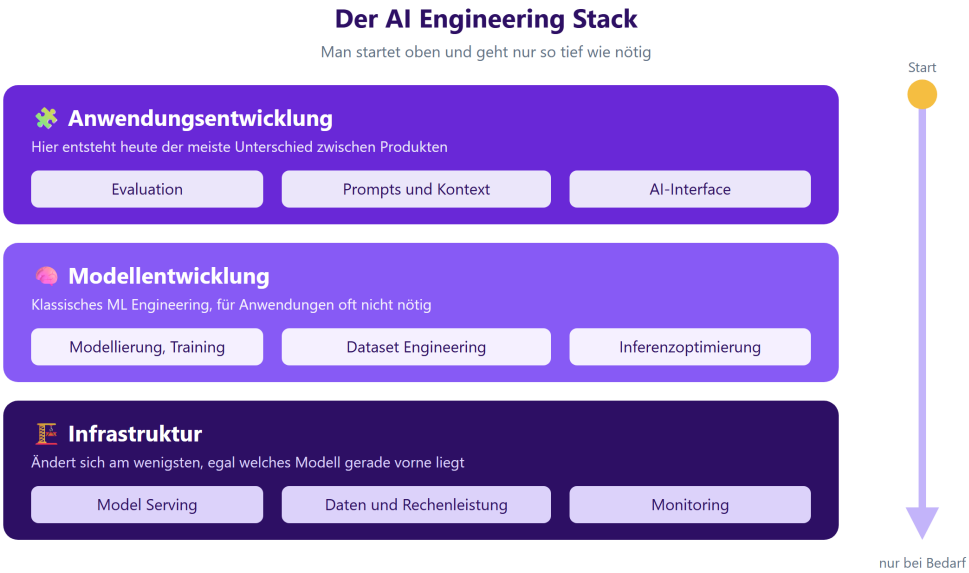


Abbildung 4: Der AI Engineering Stack nach Huyen. Entwickelt wird von oben nach unten, nur so tief wie nötig.

- **Anwendungsentwicklung:** Prompts und Kontext, Evaluation und die Oberfläche für die Nutzer. Weil viele Teams dieselben Modelle nutzen, entsteht hier heute der meiste Unterschied zwischen Produkten.
- **Modellentwicklung:** Modellierung und Training, Dataset Engineering und Inferenzoptimierung. Das ist das klassische Feld des ML Engineering. Für viele Anwendungen braucht man diese Schicht nicht, Inferenzoptimierung wird aber wichtiger, weil große Modelle teuer und langsam sind.
- **Infrastruktur:** Model Serving, Verwaltung von Daten und Rechenleistung, Monitoring. Diese Schicht ändert sich am wenigsten, egal welches Modell gerade vorne liegt.

Man beginnt oben und geht nur so weit nach unten, wie nötig. Huyen beobachtet außerdem, dass AI Engineering dem Full-Stack-Engineering näher rückt: Früher kamen erst Daten und Modell, dann das Produkt. Heute kann man mit dem Produkt beginnen und erst in Daten und Modelle investieren, wenn sich die Idee bewährt.

Für diesen Kurs heißt das:

- **Anwendungsentwicklung** ist der erste Schwerpunkt. Ihr lernt, ein Modell mit guten Prompts und dem richtigen Kontext zu versorgen, Wissen aus Dokumenten abzurufen, Agenten mit Werkzeugen zu bauen und die Qualität per Evaluation zu messen (VL 2 bis 8).
- Aus der **Modellentwicklung** nehmen wir nur die Inferenzoptimierung mit: Quantisierung, Batching und Caching, damit ein Modell schneller und billiger antwortet (VL 10). Modelle trainieren wir nicht.
- **Infrastruktur** ist der zweite Schwerpunkt. Ihr verpackt euer System in Container, betreibt es in Kubernetes mit eigenem Model Serving, schützt es mit einem Gateway und überwacht es im Betrieb (VL 9 bis 13).

★ Der Kurs geht genauso vor, wie Huyen es für AI Engineering beschreibt: Zuerst steht ein laufender Modellaufruf, alles andere wächst Schritt für Schritt dazu.

Was heißt cloud-native?

Die Prinzipien

Die **Cloud Native Computing Foundation** (CNCF) ist eine gemeinnützige Stiftung unter dem Dach der Linux Foundation, gegründet 2015. Sie betreut zentrale Open-Source-Projekte wie Kubernetes und Prometheus und prägt damit, was „cloud-native“ heißt. Sie beschreibt cloud-native Technologien so: Sie befähigen Organisationen, skalierbare Anwendungen in modernen, dynamischen Umgebungen zu bauen und zu betreiben, ob in öffentlichen, privaten oder hybriden Clouds. Die typischen Bausteine sind:

- **Container:** Die Anwendung wird mit allen Abhängigkeiten verpackt und läuft überall gleich.
- **Unveränderliche Infrastruktur:** Statt einen laufenden Server zu ändern, ersetzt man ihn durch eine neue Version.
- **Deklarative APIs:** Man beschreibt den gewünschten Zustand („drei Instanzen dieses Dienstes“), und das System sorgt dafür, dass er eintritt.
- **Microservices:** Die Anwendung besteht aus kleinen, unabhängig auslieferbaren Diensten.
- **Observability:** Das System liefert Logs, Metriken und Traces, um sein Verhalten von außen zu verstehen.

Die bekannteste Plattform, die diese Ideen umsetzt, ist Kubernetes. Wir arbeiten ab VL 9 damit.

Cloud-native AI

> **Cloud Native Artificial Intelligence (CNAI) –** Ansätze und Muster, um AI-Anwendungen und AI-Workloads nach den Prinzipien von Cloud Native zu bauen und zu betreiben (CNCF AI Working Group, 2024).

Das klingt nach einer einfachen Übertragung bekannter Ideen. AI-Workloads stellen die Prinzipien aber auf die Probe:

Klassischer Webservice	AI-System
Startet in Sekunden	Ein Modell mit vielen Gigabyte muss erst geladen werden
Läuft auf jeder CPU	Braucht oft eine GPU, die knapp und teuer ist
Gleiche Eingabe, gleiche Ausgabe	Ausgaben variieren, Qualität ist schwer zu messen
Kosten pro Anfrage gering	Kosten pro Token spürbar

Das CNCF-Whitepaper ordnet die offenen Herausforderungen entlang des Lebenszyklus: Datenaufbereitung, Modelltraining, **Model Serving**, Nutzererfahrung und Querschnittsthemen wie Ressourcen, Kosten und **Observability**. Für unseren Kurs sind vor allem Serving und die Querschnittsthemen relevant, weil wir Modelle nicht trainieren, sondern betreiben.

Das Kursbeispiel: der Campus-Assistent

Damit die Vorlesungen zusammenhängen, begleitet uns ein durchgehendes Beispiel: ein **Campus-Assistent** für die HTWG. Er beantwortet Fragen von Studierenden zu Prüfungsordnung, Modulen und Mensa. In jeder Vorlesung bekommt er eine Schicht dazu:

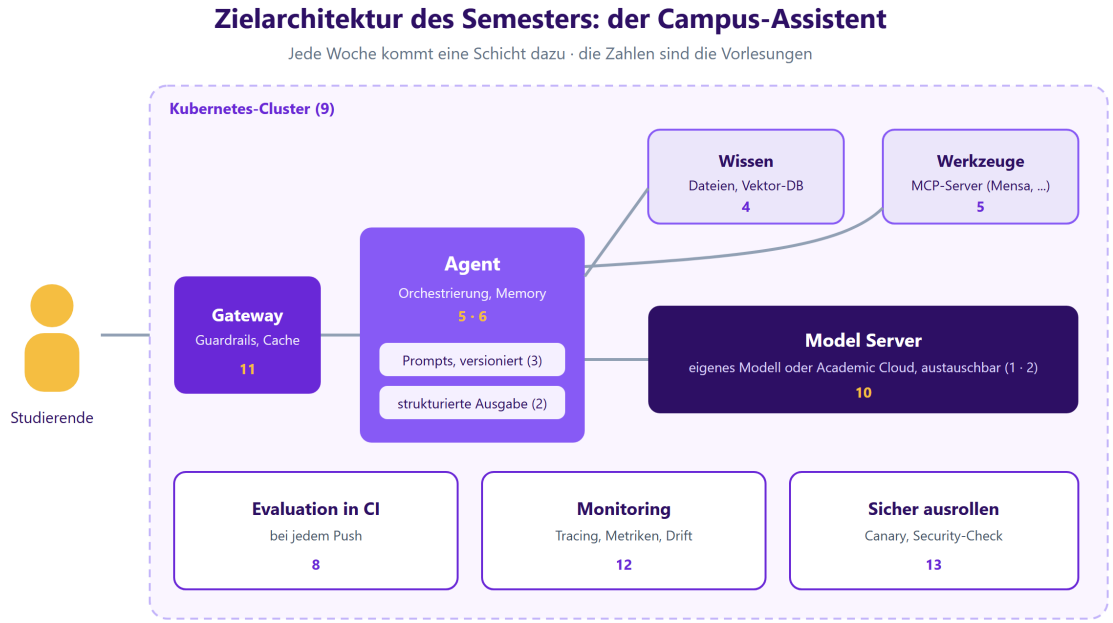


Abbildung 5: Zielarchitektur des Semesters. Die Zahlen nennen die Vorlesungen, in denen die Schicht dazukommt.

Am Anfang steht nur ein Modellaufruf. Dann kommen strukturierte Ausgaben, versionierte Prompts, Wissen aus Dokumenten, Werkzeuge und ein Agent dazu. Ab der zweiten Hälfte messen wir die Qualität automatisch, betreiben das Ganze in einem

Kubernetes-Cluster mit eigenem Model Server, schützen es mit einem Gateway und überwachen es im Betrieb.

Euer Praktikumssystem folgt demselben Aufbau. Den genauen Funktionsumfang jeder Abgabe gibt es in Moodle.

Die Übung heute

In der Übung baut ihr die unterste Stufe: eine kleine Anwendung, die ein Modell über eine **OpenAI-kompatible API** aufruft. Diese Schnittstelle ist zum Standard geworden. Lokale Werkzeuge wie Ollama bieten sie ebenso an wie die Academic Cloud der GWDG, die ihr mit eurem HTWG-Account nutzen könnt. Deshalb funktioniert derselbe Code mit beiden, nur die Konfiguration ändert sich:

```python
from openai import OpenAI

client = OpenAI(
    base_url=os.environ["LLM_BASE_URL"],
    api_key=os.environ["LLM_API_KEY"])

antwort = client.chat.completions.create(
    model=os.environ["LLM_MODEL"],
    messages=[{"role": "user",
               "content": "Was gibt es in der Mensa?"}])
```

Der Code macht drei Dinge:

1. `OpenAI(...)` baut den Client. Wohin er spricht (`base_url`) und mit welchem Schlüssel (`api_key`), kommt aus Umgebungsvariablen.
2. `create(...)` schickt die Nachricht an das Modell und liefert die Antwort zurück.
3. `model` legt fest, welches Modell antwortet. Auch das steht nur in der Konfiguration, in der Übung in der Datei `.env` .

Wer das Modell wechseln will, ändert also nur `.env` , nicht den Code.

Ihr messt dabei eure erste Time to First Token und vergleicht ein kleines lokales Modell mit einem größeren in der Cloud.

Typische Missverständnisse

⚠ **Achtung:**

- „Die Demo läuft, also sind wir fast fertig.“ Der Weg von 80 auf 95 Prozent dauert oft länger als der Weg dorthin.
- „Ich habe das Modell mit meinen Beispielen trainiert.“ Beispiele im Prompt sind Kontext, kein Training.
- „Das Modell steht im Code.“ Dann wird jeder Modellwechsel zur Codeänderung. Modelle gehören in die Konfiguration.
- „Cloud-native heißt, es läuft in der Cloud.“ Gemeint sind Prinzipien wie Container, deklarative Konfiguration und Observability. Ein Cluster auf dem eigenen Laptop kann genauso cloud-native sein.

Zusammenfassung

★ AI Engineering baut Anwendungen auf Foundation Models. Anders als im klassischen ML Engineering passt man fremde Modelle an, statt eigene zu trainieren, arbeitet mit größeren Modellen und muss offene Ausgaben bewerten. Vor dem Bauen klärt man die Rolle von KI und Mensch und legt eine Nützlichkeitsschwelle aus Qualität, Latenz und Kosten fest. Der AI Engineering Stack hat drei Schichten: Anwendung, Modell und Infrastruktur. Cloud-native AI überträgt Container, deklarative Konfiguration und Observability auf AI-Workloads, die durch Modellgröße, GPUs und schwer messbare Qualität besondere Anforderungen stellen.

Kontrollfragen

1. Nennt die drei Unterschiede zwischen AI Engineering und klassischem ML Engineering und erklärt für einen davon, welche Folge er für die Arbeit im Team hat.
2. Eine Kollegin sagt: „Ich habe das Modell mit 20 Beispielen im Prompt trainiert.“ Was stimmt daran nicht?
3. Ordnet den Schichten des AI Engineering Stacks zu: Prompt schreiben, Modell quantisieren, Antwortzeiten überwachen, Testfragen für die Bewertung sammeln.
4. Ein Assistent soll Studierenden automatisch Prüfungsfristen per Push-Nachricht melden. Ist das reaktiv oder proaktiv, und was folgt daraus für die Qualitätsanforderung?

5. Nennt zwei Gründe, warum ein AI-Dienst schwerer in Kubernetes zu betreiben ist als ein klassischer Webservice.

Quellen

- Chip Huyen: *AI Engineering. Building Applications with Foundation Models.* O'Reilly, 2025, Kapitel 1.
- CNCF AI Working Group: *Cloud Native Artificial Intelligence.* Whitepaper, 2024. cncf.io